



AFLEARN-AFEP

Analysis of African Education Data for Policy and Practice: STATA Guide

23 – 27 February 2026

TABLE OF CONTENTS

Lab Module 1: Getting to know your data in stata	3
1. Getting started in Stata	3
2. Looking at some of the data, and how it is captured in Stata.....	4
3. Simple summarising commands.....	5
4. Finding a unique identifier	8
5. Examining variables	10
6. Examining variables further	12
7. Quick introduction to do files	14
8. Missing data codes	15
9. Creating new variables	20
10. Extensions to creating new variables.....	22
11. Using complex survey data: the svy suite	26
12. Weights	28
13. Describing data: plots and tables	31
List of stata commands introduced in Module 1	38
Lab Module 2: Introduction to Regression Analysis in Stata.....	39
1. Running a regression	39
2. Interaction terms in regressions.....	43
3. A new example: returns to education	44
4. Logging a dependent variable	46
5. Education as a continuous variable	46
6. Try it yourself.....	48
7. Same data, different question.....	48
Exporting results.....	54
List of stata commands used in this lab	55
Module 3: Replication Lab	1
List of stata commands used in this lab	6

Lab Module 1: Getting to know your data in STATA

OBJECTIVES: By the end of this lab, you should have:

1. Learned some of the foundations of using STATA
2. Gained some skills related to getting to know your data and preparing it for application of a research design

1. GETTING STARTED IN STATA

STATA is one type of statistical program that economists use to manage, look at and analyse economic data. It is particularly suited to the analysis of complex survey data – in our case, data from education assessments. It has a library of pre-programmed commands, which are useful for creating summary statistics from the data, and for performing more detailed statistical analyses. STATA is also programmable (we can write programs to tell STATA to perform a specific task), but we will not be learning how to program in STATA in this course.

Open up STATA by navigating to

→ Start | Programs | STATA

You should now have the STATA window open. Note that there is a set of pull-down menus, as well as 4 smaller windows, named: **History, Variables, STATA Command, and STATA Results**. When we want to tell STATA to do something (execute some command), we will type it into the STATA Command line window. The results of our command will show up in the STATA Results window. In any session, all commands that we give to the program will show up in the History window once we execute them. The History window is useful, because it allows us to recall commands quickly (by clicking on the relevant command in the History window it will reappear in the STATA Command line) and can be saved for editing later into a do-file.¹

Right now the Variables window is empty – this is because we have not loaded any data. Whenever you have a data set open, the variable names will show up in this window.

¹ A STATA do-file is a short program file, which runs a set of commands at once. We will learn about these later.

2. LOOKING AT SOME OF THE DATA, AND HOW IT IS CAPTURED IN STATA

You will need to download the data from MS Teams before you can open it in STATA. Double click on mics6.zip in the folder Day 2 / MICS6 , unzip the file and save it to your desktop or somewhere else convenient.

Open up the data set, by choosing the FILE drop down menu, and then choosing OPEN and selecting the file you downloaded from MS Teams. You should now see the list of variables appear in the Variables window. Do you notice a line has appeared in your History window starting with the word **use** and then a file path?

Let's go over again how to load data because you will do this many times. First, remove the data by typing in the command line:

```
clear
```

The variables are gone in the Variables window. You can reload your data in two ways. Either click on the **use** command that appeared in your History window. Or, type in the command line replacing the italics portion with the file path to the folder where the data is stored. Note the quotation marks:

```
use "yourfilepath\ipums_mics6_africa.dta"
```

The dataset we are using is a harmonised version of several MICS surveys. MICS stands for the Multiple Indicator Cluster Survey and is a household survey run by UNICEF in many different countries around the world with a focus on health and education outcomes. If you go to the MICS website (<https://mics.unicef.org/>) and navigate to the surveys tab, you will only find individual country datasets available for download. The dataset that you have now loaded into STATA was created for this course using STATA code from IPUMS (<https://www.ipums.org/>), a research unit that focuses on data harmonisation and integration. Specifically, the dataset was created using code from the IPUMS-MICS project (<https://mics.ipums.org/mics/>) that allows us to create a single dataset from many separate country datasets with variables that all have the same names and value codes (harmonised).

First let's take a more detailed look at what variables are stored in the data set. In the STATA command line, type in:

```
describe
```

You should see a list of what variables are contained in the data, how many observations there are, how many variables there are, what kind of variables they are (eg. 'byte', 'float', 'int', 'str' or 'long' variables), and any comments which are attached.²

Now type in:

count

This command counts the number of unique observations in your dataset. This information (the number of variables and observations) can also be seen in the Properties window in STATA.

Next, type:

browse

This command directs you to a spreadsheet, where the data appears. You should note the following:

- ➔ each OBSERVATION (in this case, each child between the ages of 5 and 17 in the household for which there is recorded information) appears on a separate ROW of the spreadsheet. If you move along the row, you can see the rest of their data recorded for each question. For example, the first person in the dataset lives in Algeria, in 2018-9 in cluster 1, hh number 7 and if you keep scrolling across you will see that she is a girl who is aged 14 and living in an urban area.
- ➔ each VARIABLE appears in a separate COLUMN of the spreadsheet. What is the age and gender variable called?

Using the **browse** command, find the 420th observation. What is this child's age? Their gender?

3. SIMPLE SUMMARISING COMMANDS

A frequency distribution table lists all the observed values for a given variable and the number of observations that take on each of these values. **tabulate** produces one- and two-way tables of frequency counts. In this first class, we will use this command

² If you see **--more--** at the bottom of your screen, you need to press the space bar to continue scrolling.

to produce summary tables of data, to answer the general question: how much of our data set falls into different categories of each variable. We can see what **tab** does easily by example. We know our MICS dataset has more than one country so we can type the following to find out a little more about our dataset:

```
tab country
```

How many observations are from Lesotho?

If you are interested in exactly how a particular command works, you can always use the **help** function. Type:

```
help tab
```

and click on **tabulate oneway**. This will tell you all about the different kinds of tab commands.

Similarly, we can describe the time dimension of our dataset by typing:

```
tab year
```

This tab might not look very readable if the number labels are on. A useful command to remove number labels is:

```
numlabel, remove
```

Now tabulate country again and see what it looks like:

```
tab country
```

Hopefully more readable. In other circumstances you might want to see the number labels so it is helpful being able to turn the labels on and off. You can add the labels back again by typing:

```
numlabel, add
```

The tab command is also able to produce two-way tables. We can see which countries were surveyed in which year by typing:

```
tab country year
```

Which country appears more than once in our dataset? Consider another two-way table:

tab country urban

BOX: TAB WITH ROW, COL, CELL

The **tab** command is versatile for describing data. If we are interested in the percentages, we can specify which type of percentages we want:

tab country urban, row will give a frequency table which generates the percentage of the ROW observations which are in each COLUMN category (i.e. the percentages across each row sum to 100). Thus, 67.63% of the learners from Algeria in our sample live in urban areas. At the bottom of each column, note that you will see the percentage of the entire data set that fall into that particular column category.

However, this table might not be very readable and you might only be interested in percentages and not want to see the frequencies. In which case, type:

tab country urban, row nofreq

This is much more readable. Similarly, **tab** can be extended in the following ways (with or without the **nofreq** option):

tab country urban, column nofreq will give a frequency table which generates the percentage of the COLUMN observations which are in each ROW category. Thus, 17.611% of learners living in urban areas in our sample come from Algeria. At the end of each row, you will find the percentage of the entire data set that fall into that particular row category.

tab country urban, cell nofreq will give a frequency table which gives the percentage of the sample in each possible cell. For example, 8.4% of the sample are Malawian learners from rural areas.

To learn more about two-way tables, remember you can always use the help file:

help tab

And this time click on **tabulate twoway**.

4. FINDING A UNIQUE IDENTIFIER

An important first step in getting to know your dataset is finding out the unit of analysis and what uniquely identifies observations. Remember when we used the **count** command, we saw there were 184,190 observations in this dataset. It is important to know which variable or combination of variables will uniquely identify these 184 thousand observations and what each row represents (a person, a classroom, a school?) when we **browse** our data. If we **describe** our data again, we see the IPUMS-MICS approach usefully includes a variable called unitanalysis. Type:

```
tab unitanalysis
```

This tells us that the unit of analysis in this dataset are children aged 5-17 years. Typically, datasets won't include such a convenient variable. However, it is more normal for datasets to include some type of "ID" variable or "unique number" variable. The way to confirm this is by using the **isid** command. This command confirms whether the variable/s you list after it uniquely identify your observations. Some investigation of the IPUMS and MICS documentation suggested a combination of variables called sample, hhno and linech should uniquely identify each child in the dataset.

Use the **codebook** command to learn about what each of these variables are:

```
codebook sample hhno linech
```

This command gives you information about each variable. It tells you the variable type (string, numeric), the variable label; how many unique values there are; the range of these values; the number of missing observations; value label information, and the numeric range. How many samples are there in this dataset? As you can see there are only 80 unique household numbers and 27 unique lines within households. These hardly seem enough to uniquely identify 180 thousand children, but this can be the case when we combine these variables.

Try:

```
isid sample hhno linech
```

What do you find? We get an error telling us we have not yet found the right combination of variables! Some more investigation suggests we also need to include the cluster variable. What is the cluster variable:

```
codebook cluster
```

Try **isid** again including this variable:

```
isid sample cluster hhno linech
```

No news is good news with **isid**. If there is no error output, it means you have a combination of variables that uniquely identify the children in the dataset.

In order to make it easier to browse the dataset, we are going to create a single unique identifier variable. We will go over creating variables again later in more detail, but we are jumping ahead for this one step here because it is so helpful to have an ID variable. To do this we are going to use an **egen** command which we will cover again later. For now, all you have to understand is that this command creates a single unique identifier from the combination of variables we have identified using **isid**. Type:

```
egen childid = group(sample cluster hhno linech)
```

```
codebook childid
```

```
isid childid
```

The **codebook** shows you that there are as many unique values of our new **childid** variable as there are observations in the dataset. We can see this is a numeric variable that ranges from 1 to 184 190; there are no missings; and the **isid** command confirms this variable works as a unique identifier.

When browsing, it is often easiest to read the dataset if the unique identifier is the first column. To change the order of your variables in the dataset and the browse window, use the order command. Typing the following command will place **childid** as the first variable in our dataset:

```
order childid
```

We can confirm this by typing:

```
desc
```

```
br
```

5. EXAMINING VARIABLES

Often, we are interested in whether a dataset contains a certain variable and reading a long list of variables from the **desc** command can be tedious. There is an efficient way to find the names of variables that you are interested in. Imagine you are interested in studying the effect of private schooling on numeracy scores. The first variable you want to find is one about school type (private or not). Type in:

```
lookfor type
```

This should give you a list of all the variables which have 'type' in their name or label. We see there is a variable called schooltype. Let's tab this variable to see if it's what we are looking for:

```
tab schooltype
```

Great, this first look suggests this is a variable we might be able to use for our research. Next, we are interested in numeracy scores. Let's **lookfor** any words that might bring up such a variable:

```
lookfor number
```

This brings up variables like flbigger1 and flnumadd1. Do you notice that they all have "fl" in the prefix? A common practice by dataset processors (or harmonisers) is to use prefixes or suffixes for variables in the same module of the questionnaire or if they all apply to a particular unit of analysis. This is therefore a hint. Type:

```
lookfor fl
```

Now we see a full range of assessment variables including some for numeracy that don't include the word "number" that we missed the first time around, like flrecognize1. Let's use the **browse** command to take a closer look at these variables. **A useful trick when you have similarly named variables with numbers in the name is linking them with a hyphen or using an asterisk so you don't have to type them all out.** We are going to browse our unique identifier and all of these numeracy variables:

```
browse childid flrecognize1-flrecognize5 flbigger* flnumadd* flnumseq*
```

You can see that some set of children have missing data for the tests. In STATA, missing data is indicated with a period '.'. How did child 17 502 do in the numeracy tests? You can either see this by scrolling down in the **browse** window, or by typing:

```
browse childid flrecognize1-flrecognize5 flbigger* flnumadd* flnumseq* if childid == 17502
```

BOX: IF STATEMENTS

The **if** is a very intuitive way to limit your data to particular groups of interest and can be added to almost any command (including the **tab** command we've already used). The **if** is combined with comparison or logical operators. These operators are listed below:

Comparison operators		Logical operators	
==	Equal to		or
~= and !=	Not equal to	&	and
>	Greater than	~	not
<	Less than		
>=	Greater		
<=	than/equal to		
	Less than/equal to		

Let's practise interpreting two-way **tab** commands with conditional statements. Interpret the first cell of each of the following:

```
tab age sexfs if urban == 1, r nofreq
```

```
tab age sexfs if urban == 2, col nofreq
```

```
tab age sexfs if urban == 1, cell nofreq
```

Use your knowledge of the commands we have learned so far to answer the following questions:

What percentage of the sample from Algeria lives in urban areas?

What percentage of 15-year-old boys live in rural areas?

What percentage of the sample are girls living in Malawi?

6. EXAMINING VARIABLES FURTHER

Imagine you are interested in the distribution of age for Grade 2s. Use the **lookfor** command to find variables relevant to age and grade. Type:

```
tab edgradecurr
```

This looks like the right variable for grade. What about age?

A very useful command for describing your data is the **sum** command which provides sample summary statistics. Type:

```
sum age
```

Here we can see the mean, standard deviation, min, max and number of non-missing observations. What is the average age of children in this dataset? What is the standard deviation? What is the answer when we limit this to Grade 2s? Here we use a conditional statement like we have just learned about to limit the **sum** command to only children in Grade 2. Type:

```
sum age if edgradecurr == 2
```

Again, we want to go back to the trusty **tab** command since it provides a table showing the *distribution* of age in Grade 2 which can be particularly helpful when trying to get to know your data. This is because any suspicious parts of the data jump out in a more obvious way when looking at distributions than just high-level summary statistics. Let's type:

```
tab age if edgradecurr == 2
```

Comparing the **tab** and **sum** results illustrates another good reason to look at distributions. Although the mean of the distribution as reported by **sum** is 10 years, the **tab** shows that most children in Grade 2 cluster around the ages of 7 and 8 years. Why is the mean so much higher than 7 and 8 years old then? Let's inspect the distribution again but this time use a histogram to inspect it visually. A histogram is an appropriate way to describe age because it is a continuous variable. Type the following code:

```
hist age if edgradecurr == 2
```

Hmm, this is not a great plot. The bars are very skinny and not evenly spaced out. If you look at the output in the STATA command window you see (bin=44, start=5,

width=.27272727). This tells you STATA created 44 bins of a width of 0.27 into which to divide the age data and started from a value of age 5. It would be much more helpful if age was divided into bins that were a width of 1 because age ticks up one year at a time. In other words, although age is a continuous variable, it is a type of continuous variable called 'discrete'. It increases continuously in 'chunks' as opposed to smaller, more continuous, fractions. An example of a more continuous variable is income or price which can increase by the cent or the 0.01 decimal point.

The help file on **histogram** will tell you more about how to plot a histogram that is appropriate for your data. Since we have discrete data, we can tell STATA this and it will produce a histogram that uses discrete bins of one unit instead of some fraction of more or less than one (that would be appropriate for more continuous data). Type:

```
hist age if edgradecurr == 2, discrete
```

This plot looks a lot better. And also notice that in the STATA output window STATA tells you it has produced this histogram with bins of width 1. This plot is evenly spaced, and each bin clearly represents one year of age.

Now that we have a readable plot, let's return to our original question, which was about why the mean of the age distribution was so much higher than where the main share of children were clustered. What do you notice about this plot? There is heaping both around the ages of 7 and 8 and also higher up in the distribution. We could call this a bimodal distribution because it has two peaks. Does this seem right to you that are so many 12-17 years olds in this distribution relative to 10- and 11-year olds?

One way to sense-check a pattern like this is to explore to what extent it might exist across other dimensions. Using the **hist** command, we can see whether this pattern exists across countries by using the **by()** option. Type:

```
hist age if edgradecurr == 2, discrete by(country)
```

With some exceptions, this bimodal shape appears to exist across most of the countries in this dataset. What about different grades? To test this, we need to change our conditional statement from limiting to just Grade 2s to limiting to children with non-missing Grade information. To do this we limit our sample to those with a value of edgradecurr less than 96 since it is from code 96 upwards that missing codes occur. We will return to missing data again later on. For now, type:

```
hist age if edgradecurr < 96, discrete by(edgradecurr)
```

What do you notice when looking across grades? The bimodal shape is more prominent in earlier grades (e.g. 1-4) than later grades (e.g. 5+). Take a closer look at the MICS household questionnaire in order to check whether we are using the edgradecurr variable correctly. It turns out we are not quite getting this right. The question first specifies education level and then grade, meaning we need to use a combination of a level and grade variable. The variable we need is edlevelcurr. Let's look at this variable and how it corresponds with edgradecurr. Type:

```
tab edlevelcurr  
tab edlevelcurr edgradecurr
```

What do you notice? Grade 2 counts as primary in the level variable. Most children with edgradecurr == 2 are clustered in the primary level. But there are a bunch of children who have a value of 2 in the grade variable but are in various secondary schooling levels. These children are older than those in primary levels. Let's use this information to more accurately capture Grade 2s when plotting the age distribution. Type:

```
hist age if edgradecurr==2 & edlevelcurr == 200, discrete
```

Now we have a nicely-shaped bell curve with only one peak and much lower density at older ages. This exercise has hopefully demonstrated the usefulness of inspecting distributions to learn about your data and the importance of checking questionnaires.

7. QUICK INTRODUCTION TO DO FILES

Open STATA and type the following commands

```
clear all  
use "yourfilepath\ipums_mics6_africa.dta"  
numlabel, add  
tab country urban
```

Often, we will want to execute commands repetitively in STATA, or we'll want to save the set of commands for a particular procedure so that we can run them on a different data set or replicate results in the same dataset. We can write simple programs in STATA's do-file editor that will run a set of commands at once.

As an example, suppose we wanted to repeat the process of looking at the variables above. We can save the commands we have already used using the History window. Select all the commands in the History window, Right click and select "Send to Do-file

Editor”. This will open up the do file editor and a do file with all the commands you have used in your STATA session. You can now save this do file by selecting File>Save within the do file editor. Save the do file to your home directory and call it “Example.do.” Close the do file editor.

Open the do file editor, clicking on the envelope-button on the toolbar. Open the file you just created called **example.do**.

In the do file editor, click on the button with the arrow that says “Execute.” If you just want to run some of the commands in a do file then highlight those commands and click on the “Execute” button. Ctrl+L is a shortcut that highlights a line in Windows and Ctrl+D executes the highlighted command (or entire do file if no line is specially highlighted).

You can copy and paste and write new commands in the do file, and then save the do file to run again on this data set or other data sets. If you made any mistakes in your STATA session the do file will end and an error message will appear. You will need to edit the do file by correcting or deleting invalid commands. Note that if you generate new variables in a do file, you cannot run the do file again without dropping those variables or loading the data set again.

8. MISSING DATA CODES

Let’s return to our research question, which was about private school and numeracy scores. To study this question, we want to learn a little more about the schooltype variable. Let’s describe this variable with some of the commands we have already learned. Type:

```
tab schooltype
```

Tabbing schooltype gives us a break down of which children are in which type of school. The value code 10 is for Government / public school. The value code 20 is for Religious / faith school. The value 30 is for Private school which is the type we are interested in. How many children are in private school? And there are some other options, including codes 97-99 which stand for Don’t Know, Missing, and NIU (Not in universe). If we try to **sum** schooltype we get something like this:

```
sum schooltype
```

The mean of schooltype is 39.9. What does this mean? Can you interpret this? The reason you cannot interpret this like we could the **sum** for age is because schooltype is a categorical variable.

BOX: TYPES OF VARIABLES

Let's take a moment to review different types of variables. The statistical and graphical tools used to understand the distributions of the various types of variables are quite different, so it is important to understand the differences between these measures.

CONTINUOUS VARIABLES

Continuous variables have an infinite number of possible values that fall between any two observed values. For example, consider age. In our data, age is recorded in years. But it could have been recorded in months, days, minutes, or even seconds. A continuous variable is ordinal in the sense that its values have an inherent order. In the age example, an age of 16 years is one year older than the age of 15 years, thus the unit of measurement in between these two values is itself meaningful. (This may seem like common sense, but when we consider categorical variables, this will no longer be true). Examples of continuous variables in the MICS data set include age (agefs) and wealth index score (wscore). Recall when we discussed age above, we also differentiated between truly continuous variables (like price, income or in MICS, the wscore) and discrete variables (like age recorded in years) that tick up in larger, more distinct units.

CATEGORICAL VARIABLES

These are made up of separate and distinct categories which do not have an inherent order. To code these variables, each category is typically assigned a value, but this assignment is arbitrary. Take for example the schooltype variable. Each learner is assigned an arbitrary value used to indicate whether they went to government school or private school. But the value of the number (e.g. 10 or 30) is meaningless. This is why you cannot calculate the mean of a categorical variable in the same way as you can for a continuous variable. Another example of a categorical variable is the home language variable, langhome, and the gender variable, sexfs, where boys are coded 1 and females are coded 2.

There are two special cases of categorical variables. The first is categorical variables where the order does matter, but it is still incorrect to treat them like continuous variables and calculate a mean using **sum**. These are called *ordinal* categorical variables and an example is age group, agefs_grp. For the age-grp variables, lower values correspond to younger ages meaning higher values of the agefs_grp variable

is meaningfully connected to age itself. But it would still not make sense to say the mean of this variable is 2 – what does that really mean? Don't confuse ordinal categorical variables with continuous variables.

Another special case of a categorical variable is a dummy variable, also called an indicator variable. Dummies only take on the values of one or zero. For example, a dummy for girls would be coded 0 for boys and 1 for girls. As we will show shortly, you can interpret the mean from a sum command when using a dummy. But the interpretation changes from a *mean* to a *share*. The mean for a dummy for girls for example, will be the share of the sample that are girls.

Let's look again at our frequency table for schooltype instead:

tab schooltype

An important part of working with data is knowing how to deal with missing data. Generally, we would call codes 10-52 “non-missing” and say 69.27% of the sample have “non-missing” data for schooltype. But there are different types of missing data in schooltype and it's important to know where they all come in order to assess the quality of your variable.

Some people were asked the question of what type of school a child attended but didn't know the answer – they got a code 97. This was for 49 children. Sometimes people can also refuse and get a separate code for that. This is called “non-response”, and we will talk more about this later because if non-response is very prevalent, it can undermine the quality of your data. In this case, only 0.03% of the sample did not respond so we do not have to worry about this.

A further 15 children just have schooltype information missing in some unspecified way and get a code 98. There are different reasons why the data is missing in this way that we cannot tell just by looking at it. But, for example, maybe the enumerator accidentally skipped that question or maybe a mistake was made during data entry or post-processing. Often, this type of missing data can be ignored, partly because it is meant to be due to random errors and also partly because we truly have no information about why it is missing (this isn't a good reason, but a pragmatic one). Similar to non-response though, if very high proportions of your sample are missing this can undermine the quality of your data and the extent to which it is fit for the purpose of answering your research question.

Something to notice about this type of missing data is it does not always crop up in a value code, sometimes it is just coded as a period as we saw previously. Do you notice that the total number of observations in the tab is less than that in the whole dataset

(if you have forgotten how many there are in the whole dataset, use **count**). The **tab** command allows us to add these extra missings to our table using the option **m**. Type:

```
tab schooltype, m
```

This table now has the total number of observations in the whole dataset and reveals an additional 24 950 children with true unspecified missing information for schooltype. This is now more than 10% of our sample, so we should investigate a bit why there is so much missing data. We will do this shortly.

The third missing code is NIU (not in universe) and a large number of 48 863 children, or 26.53% of the sample, fall into this category. NIU means a child was not eligible to answer that question for some reason, e.g. the foundational learning module in MICS was only asked of children aged 7-14 years so younger and older children in the dataset will be NIU for variables in that module. NIU can also be coded if there was a skip code in the questionnaire. An example of a skip code would be first asking whether a child attends school and then – if the child is not attending school – skipping all the subsequent questions asking about the characteristics of the school and going straight to the next module, for example, on child health. A skip code happens when the answer to a particular question means the respondent might not have to answer the next set of questions because they do not apply to their situation and can instead be filtered to questions later on in the questionnaire or a set of questions specifically for their situation.

Because NIU means asking about school type was not applicable to these 48 863 children, this is an acceptable reason to ignore this type of missing data. If, on the other hand, we had 26% of the sample falling into non-response or missing, that would be a much bigger problem to confront for the quality of the schooltype variable. Nevertheless, it can still be important to understand who has missing and non-missing data for your variable. This is because later we are going to create a 'clean' version of this variable that omits all the missing codes. Knowing who you are omitting tells you who is and isn't in your sample and this can have repercussions for how you frame your research question and interpret your results.

Let's investigate who is and isn't in the schooltype variable. A lesson we just learned was the importance of checking our questionnaire. So, let's check the questionnaire to see if there are any skip codes. It turns out there are. Only children who attend school are asked this question about school type. So, let's find the attendance question.

```
lookfor attend
```

And see if this explains the NIUs, for example. Type:

```
tab schooltype schoolcurr
```

From this table, we can see that yes, indeed, about 12 500 children who are not currently attending school have been classified as NIU. But there are still another 8 900 who are attending school and who still fall into NIU and a further 27 000 children who are NIU for both variables. Being NIU for schoolcurr is another hint that there might be a prior skip code so let's look at the questionnaire again. Indeed, prior to being asked if a child currently attends school, households are asked if the child has ever attended school. If they answer negatively, the enumerator does not have to bother to ask them if they are currently attending school, because they have effectively already said they are not. From our previous **lookfor** we know the relevant variable is schoolever. Type:

```
tab schooltype schoolever
```

Indeed, almost exactly the same number of about 27 000 children from the previous table count as having never attended school. They therefore were NIU for all the school characteristic questions including current attendance and school type. There are about 21 400 children who have ever attended school who fall into NIU for schooltype. We know some of these are children who might have attended school before, but not currently. To show this, let's use a conditional statement. Type:

```
tab schooltype schoolcurr if schoolever == 1
```

this omits the 27 000 who have never attended school

```
tab schooltype if schoolever == 1 & schoolcurr == 1
```

The last table shows that even once we have accounted for school attendance, there are still 8 900 kids who are NIU. Who are these children? Checking the questionnaire again, we see there is also a skip code related to Early Childhood Education. Children who are in ECE are not asked questions related to school characteristics. Recall the level variable from earlier. We can type

```
tab edlevelcurr schooltype
```

The same 8 900 children who are missing schooltype fall into code 100 (LESS THAN PRIMARY), i.e. ECE. Let's try and use conditional statements to limit our **tab** to the eligible group.:

```
tab schooltype if schoolever == 1 & schoolcurr == 1 & edlevelcurr != 100
```

Great! So now we have pretty much removed all the missings and know who is in our sample: children attending school at a level greater than ECE. But don't forget that previously we used the **, m** option to show there were an additional 24 000 children missing that don't appear in a regular tab because they don't have value codes for this variable (this is already a hint at what might be going on here). Let's have a look again with the missing option:

```
tab schooltype if schoolever == 1 & schoolcurr == 1 & edlevelcurr != 100, m
```

There are still 16 700 children who have truly missing (.) information for this variable that cannot be explained by them not being in school or in ECE. Since we know there are meant to be codes for missing data, the fact that this data is totally missing suggests the question wasn't even asked at all in the questionnaire. This could be the case for particular countries. To test this theory, let's use the **tab** command to find out if missing data is confined to particular countries or spread across all the countries in the dataset. Type:

```
tab country if schooltype == .
```

Aha, only three countries have completely missing data. Let's do an extra check to see if the frequencies in this table correspond to the full samples for these countries. Type:

```
tab country
```

This confirms this is the case. We could go and check the questionnaires for Congo, Gambia and Togo to confirm, but this very much looks like this question wasn't in the questionnaires for these countries. This is a useful piece of information about our sample.

9. CREATING NEW VARIABLES

Now we want to create a "clean" version of schooltype; meaning a version we can easily use and interpret in our data analysis. More specifically, we want to create a dummy variable. Let's learn about some useful commands for creating and cleaning variables. First, we will use the **generate** command. Let's start by typing:

```
g private = 1 if schooltype == 30 this creates a variable called private that equals 1 when schooltype equals 30
```

```
replace schooltype = 0 if schooltype != 30 this replaces that same variable equal to zero when schooltype is not equal to 30 (i.e. all other cases)
```

replace schooltype = . if schooltype >= 97 but don't forget the missings, we allocate them too

Then to confirm we have created this variable properly we can do a two-way table:

```
tab schooltype private, m
```

Great! However, STATA provides a shortcut for creating dummies. Let's recreate the same variable to learn this shortcut. First, we need to drop the old variable:

```
drop private  
g private = schooltype == 30
```

This code tells STATA to create a variable called private that is equal to 1 when schooltype equals 30 and zero otherwise. So, it does two lines of code in one. Confirm:

```
tab schooltype private, m
```

Notice that the missings have also been allocated to zero. Let's clean that up:

```
replace schooltype = . if schooltype >= 97  
tab schooltype private, m
```

That looks right. Let's learn one last way to create the same variable using the **recode** command. Again, we will drop the existing private variable and recreate it with this new command. This command now takes what we did in three lines originally and reduces it to one line. We are able to code all the values including missings in one line. The **recode** command uses brackets for each value *of the new variable*. On the left-hand side of the equals sign, we put all the values from the original variable (schooltype) and on the right-hand side we put the value we want associated. Remember you can always use the **help** command if you are unsure. Type:

```
drop private  
recode schooltype (10 20 50 52 = 0) (30 = 1) (97/max = .), g(private)  
tab schooltype private, m
```

With the recode command, you do not have to use the **, g()** (generate) option. If you left that out, the command would recode the original schooltype variable. Generally, this would not be a good idea in case you actually need the original values again. Good practice is to maintain the original version and create a separate new version of the variable you need.

Now we have a dummy that indicates private school attendance for children with non-missing schooltype data and what's important is we also have a good grip on who is and is not in this sample (e.g. Gambia) and that it is the right sample (e.g. only children attending school). Now we can use the **sum** command on our dummy and confirm we can interpret this as a share (instead of a mean) by comparing it to the relevant **tab**. Type:

```
sum private  
tab schooltype if schoolever == 1 & schoolcurr == 1 & edlevelcurr != 100
```

The mean of private can be interpreted as 14% of the sample is in private school. This is exactly the same number and interpretation as the 14% for private school in the schooltype variable. Hopefully you can see how important it is to limit to the right sample, since this number will be sensitive to who is and isn't in the denominator (i.e. the number of zeroes).

Let's move on to labelling our variable so it is a bit more readable. The **lab var** command gives the variable a label that appears when you **describe** your data or use **codebook**. Type:

```
codebook private  
lab var private "Attends a private school"  
codebook private
```

Now we may also want to label the value codes (the 0 and the 1) with what they mean. This takes two steps. First, defining a label scheme and then applying that scheme to the variable. We are going to call our scheme PS. Type:

```
lab define PS 0 "Non-private" 1 "Private"  
lab val private PS
```

Now let's see if the values have labels by **tabbing**:

```
tab private  
numlabel, add  
tab private
```

10. EXTENSIONS TO CREATING NEW VARIABLES

We have gone over three ways to create a simple dummy variable. Sometimes we want to create more complicated variables. Consider a numeracy score. Let's assume you have gone ahead and read the questionnaire and used the **lookfor** command to

find the relevant variables in your dataset. You now know there are 20 different questions in the numeracy test. Our goal is to combine these 20 separate questions into a single variable capturing the score out of 20 of how many questions a child answered correctly. One way to do this is to add up 20 separate dummy variables. To begin, let's just **browse** our data again and focus on the 20 components for each child.

```
browse childid flrecognize1-flrecognize5 flbigger* flnumadd* flnumseq*
```

What you will notice is firstly there is a bunch of missing data. We will come to dealing with that. But also that the components themselves are not yet dummies we can easily add up. Let's exit the **browse** and take a closer look at the value codes of one of these components:

```
tab flbigger1
```

As you can see, flbigger1 is not a dummy and includes a few options like "inconsistent" and "no attempt". It is your job as the researcher to decide whether those codes count as 1's, 0's or missings in your dummy variable. This is a decision that must be based on the questionnaire and your research question priorities. Let's say we decide to code a correct answer as 1 and everything else as a 0 unless it is NIU. You have already learned how to do that manually for each variable doing so for 20 variables is a lot of work. A fast way to do this is using a loop. Loops are powerful tools for repetitive tasks. Think about if you were to manually type out code for 20 dummy variables created from the original variables all with similar names numbered 1 to 5. You would probably do some copy-pasting and just change the number manually (room for error!). Loops are the code version of copy-pasting and changing the number automatically. Here we are just going to learn about a **forval** loop although you get different types of loops that handle different scenarios (e.g. **foreach**) and the **help** files are a valuable resource. Here is the generic version of the **forval** loop logic from the STATA help file:

```
forvalues lname = range {  
    Stata commands referring to `lname'  
}
```

Here 'lname' is a *local*. A local is a temporary placeholder that can take on changing values that you have prespecified. Also take note of the curly braces. In order to clean our numeracy test component, we might write a loop that looks like the following (run from a do file – it's much harder to run loops from the command window):

```
forval y = 1/5 {  
    g flrecognize`y'_clean = flrecognize`y' == 1  
    replace flrecognize`y'_clean = . if flrecognize`y' >= 8 /
```

```

g flbigger`y'_clean = flbigger`y' == 1
replace flbigger`y'_clean = . if flbigger`y' >= 8

g flnumadd`y'_clean = flnumadd`y' == 1
replace flnumadd`y'_clean = . if flnumadd`y' >= 8

g flnumseq`y'_clean = flnumseq`y' == 1
replace flnumseq`y'_clean = . if flnumseq`y' >= 8

}

```

If we were to describe this loop in words, it would say: for values ranging 1-5 that the local called y can take on, generate a new variable called flrecognize(value1-5)_clean that is a dummy when flrecognize(value 1-5) equals 1 and is zero otherwise and clean up the missings. Repeat for the other components.

Let's now use these dummies we have created to make a test score variable called numscore. We can use **generate** to do this:

```

g numscore3 = flrecognize1_clean + flrecognize2_clean + flrecognize3_clean +
flrecognize4_clean + flrecognize5_clean + flbigger1_clean + flbigger2_clean +
flbigger3_clean + flbigger4_clean + flbigger5_clean + flnumadd1_clean +
flnumadd2_clean + flnumadd3_clean + flnumadd4_clean + flnumadd5_clean +
flnumseq1_clean + flnumseq2_clean + flnumseq3_clean + flnumseq4_clean +
flnumseq5_clean

```

Now let's have a first look at this variable:

```
sum numscore
```

It ranges 0-20 which is correct, but looking at the number of observations there are still quite a few missing children. Remember to check the questionnaire for skip codes and eligibility requirements. It turns out there are some of these conditions that apply to our sample here. Only children aged 7-14 years did the numeracy test and only if consent was granted from both the parent and the child. Let's **tab** using this conditional statement:

```
tab numscore if age >= 7 & age <= 14 & flconsent == 1 & flassentchild == 1, m
```

There are very few missings here that ordinarily we would not worry about. For now, let's learn a different way to make the same variable. We created this variable by

manually typing out the sum. But we could use a command from the **egen** suite of commands. **Egen** is an extension of **generate** and is very versatile. Type:

help egen

As you scroll through the help file you might notice an option called **rowtotal** that sums the values of the variable list in the bracket. That is exactly what we did! Great, we could use this command. But be careful to notice how **rowtotal** treats missings – it sets them to zero. Hmm, is that what we want? It also has the option to specify **, missing** that will set the new variable to missing when all the variables being totalled are also missing. Alright, let's create a version of numscore using **egen** and compare. Type:

```
egen numscore2 = rowtotal(flrecognize*_clean flbigger*_clean flnumadd*_clean  
flnumseq*_clean), missing
```

An advantage is we get to also use shortcuts like * to write out long variable lists. What do you notice is different though when you **tab**:

```
tab numscore2 if age >= 7 & age <= 14 & flconsent == 1 & flassentchild == 1, m
```

We have far fewer genuine missings and more zeroes. That's not great, if this method is not exactly replicating the manual sum. Can you think of what might be happening here? If you reread the **help egen** file closely, you will see that **rowtotal, missing** ignores missings but still sums up the remaining components if not all variables in the sum are missing. This means a child could get 15 questions correct and then have missing data for another five and **rowtotal, missing** will give them a score of 15/20. Does that sound right to you? The manual sum drops the child completely if they have *any* question missing. What are the relative advantages or disadvantages of these two different approaches?

We can use another **egen** command to find out if this is indeed what is happening. The **egen = missing()** command counts how many missing cases there are for each observation in the variable list in the bracket. Type:

```
egen missing_any = rowmiss(flrecognize*_clean flbigger*_clean flnumadd*_clean  
flnumseq*_clean)  
tab missing_any if age >= 7 & age <= 14 & flconsent == 1 & flassentchild == 1  
count if missing_any < 20
```

The **tab** is interpreted as meaning 95 724 children have no missing data (they are 'complete cases'). This corresponds to the total non-missing observations when we

did the manual sum creating numscore. Then 16 children have 1 of the components missing. 28 children have 2 of the components missing. Notice that 85 children have all components missing and this number corresponds to the . missings in the **tab, m** when numscore was created using **rowtotal, missing**. In other words, the difference between numscore created by manual addition (numscore) versus **egen** (numscore2) is the manual addition only keeps complete cases. Generally, this is better practice, but we have now also learned the power of **egen**.

11. USING COMPLEX SURVEY DATA: THE SVY SUITE

Often, we are interested in calculating inferential statistics from our dataset and not just sample characteristics. That is, statistics we can infer represent a true population in the outside world as opposed to just a characteristic of the data in front of us. In order to make this leap (from our desktops to the real world) we use the tools of inferential statistics, which account for how certain we are that our estimate matches the real world.

When we use the **sum** command to calculate the mean of a variable in our sample, we can be sure that the number it produces is the mean, because **sum** has all the information it needs to calculate that mean (in the form of the dataset). But, when we want to say the mean of a variable represents the mean of, for example, the Zambian population aged 5-17, we cannot be so sure. The full Zambian population aged 5-17 is not inside our computers or in our dataset. Instead, we have a sample that may be better or worse at 'representing' the Zambian population. When saying a number is the mean for this population, we need to be honest about how certain we can really be that the number in our dataset and the real world match up. Characterising this uncertainty is the role of inferential statistics.

The **mean** command estimates the mean. Compare this to the **sum** command. Type:

```
sum numscore  
mean numscore
```

Both produce the same point estimate of 10.78. But the former calculates the mean as if all the information we need is in the dataset. This is a sample characteristic. It therefore also reports a standard deviation. The latter *estimates* the mean treating the dataset as a sample of a larger true population. Therefore, we also get standard errors and confidence intervals in the output that we don't get with **sum**. These capture uncertainty regarding whether the point estimate is exactly correct. What do we mean 'treated the dataset as a sample'? Specifically, the dataset is treated as a

simple random sample that is self-weighting.³ But is this the sampling design of MICS? Properly characterising uncertainty – in order to construct standard errors and confidence intervals – requires accounting for survey sampling design.

If one is using a survey with a complex sampling design, then one needs to tell STATA so that it can correctly construct your standard errors. We tell STATA we are using complex survey data with the **svy** suite of commands (type **help svyset**). First, we **svyset** our data, which means we tell STATA what our sampling design was. Then we use the **svy:** prefix before we run an estimation command (like **mean** or **total**) in order to tell STATA to account for our complex survey design when estimating that statistic. Not every command we have learned can take this prefix (e.g. the **sum** command) because not every command is meant to supply inferential statistics.

We are now going to set up **svyset** for MICS and go through how adding each element of the complex survey design affects sampling variability. MICS is a multi-stage cluster designed survey. We are also going to learn about the design effect (DEFF). The DEFF is the ratio of a variable's variance calculated using the complex sample design to the variance calculated assuming a simple random sample (SRS). For a complex sampling design, it tells you how many more observations you require in order to match the information content of a simple random sample. Put differently, DEFF tells you what your complex survey sample size is equivalent to, had you run a SRS. The cluster variable in our dataset is called `imicscluster`. Let's start by adding the cluster. Then we will estimate the mean for `numscore`, but also another continuous variable in our dataset called `wscore` which is the household wealth index. Type:

```
svyset imicscluster  
svy: mean numscore wscore  
estat effects, deff
```

Do you notice that the standard error increases and the confidence interval gets wider for `numscore` in comparison to the results from the non-prefixed **mean** command above? This happens because clusters increase sampling error or variability by reducing the amount of statistical information you get for the same number of observations since observations within clusters are typically more similar or homogenous. In other words, to some extent you are getting more of the same type of information from the same cluster. This idea is captured by the DEFF.

For the wealth score, we have a DEFF of 6.57 and our sample is 95 833. Due to our complex sample design our precision is only as good as an SRS of $95\ 833/6.57 = 14\ 586$

³ Every observation counts equally, which is like everyone having a survey weight of 1. We will get to survey weights soon.

observations. This means our complex sample of 95 thousand observations is only as good as a SRS of 14 thousand observations in terms of precision or information content for the household wealth score. Can you calculate and interpret the design effect for numscore?

Why do you think the DEFF is higher for wscore than numscore? Numscore varies at the child-level whereas wscore varies at the household level and will be the same for children in the same household. We can also reason that household wealth is likely more homogenous within clusters than numeracy scores. We can test this idea by calculating the intra-cluster correlation (ICC) coefficient of each variable. This is a measure of how similar variables are within each cluster. Type

```
loneway wscore imicscluster  
loneway numscore imicscluster
```

The ICC is much higher for wscore than it is for numscore. When running these commands what do you notice about N in the **loneway** commands versus the **mean** commands? When you include more than one variable in the **mean** command, the N limits to the lowest common denominator. Sometimes this is appropriate and sometimes not, so just be aware of this.

Next, let's add strata to our **svyset**. Type:

```
svyset, strata(urban)  
svy: mean numscore wscore  
estat effects, deff
```

Compared to the non-**svy** mean, now the standard errors and confidence intervals get smaller. This happens because stratification is a protection against a bad sample by ensuring we have minimum sample size for different parts of the country. As such, the DEFF results are also less than one, suggesting stratification represents an improvement in precision relative to a SRS.

Finally, notice that the clusters and strata affect the size of the standard errors and confidence intervals, but the point estimate of the mean remains unchanged. This is because sampling design influences precision only.

12. WEIGHTS

The part of sampling design that does influence the point estimate, is weights. Weights are factors assigned to observations during post-processing. Typically, they vary from person to person meaning calculating a mean with and without weights

will change the point estimate. Weights play several roles in surveys. They are used to account for unequal probability of selection (e.g. in a complex survey design), they are used to account for unit non-response, and they are often 'calibrated' which means they make sure the dataset reflects certain moments from an external benchmark, like a census.

To use a weight in STATA, you include your weight variable in a square bracket at the end of your command. However, you also have to tell STATA what type of weight this is in order for it to use the weight correctly when estimating. STATA has four different weight classes – fweights, pweights, aweights and iweights. To read about these different types of weights type

help weight

BOX: TYPES OF WEIGHTS IN STATA

- fweights – frequency weights
 - represent the number of duplicated observations
 - used for grouped data
 - therefore have to be integers
 - fweight of 30 means there are 30 observations with the same values on all variables
- aweights – analytic weights
 - these are weights that are inversely proportional to the variance of an observation
 - used for heteroscedasticity corrections
 - the scale of the weight is arbitrary: it is only relative weights that matter, Stata will rescale these to sum to one
 - common use is when observations represent averages and the weights are the number of elements that gave rise to the average
 - aweight of 30 means that data is the average of 30 observations
- pweights – probability weights
 - these are sampling weights
 - these weights are the number of subjects in the full population that the sampled observation in your data represents
 - pweight of 30 means the observation had a 1 in 30 probability of being included in the sample and “represents” 30 subjects in the population

- iweights – importance weights
 - mostly used by programmers

Most STATA commands can deal with weighted data although there are some commands that do not allow all four types of weights. The syntax for producing weighted results is the same in most STATA commands: you specify the weight variable inside square brackets at the end of the command but *before the comma*. Below are some examples:

```
mean numscore [pw=weightfs]  
tab country [w=weightfs], sum(wscore)
```

The wrong weight type can produce very misleading standard errors. To see this let's estimate the mean words read correctly using fweights, aweights and pweights.

```
mean numscore [fw=weightfs]  
mean numscore [iw=weightfs]  
mean numscore [aw=weightfs]  
mean numscore [pw=weightfs]
```

Notice that the first command just gives an error. weightfs is not a frequency weight. The next three commands yield the same point estimate but the standard errors are different. Typically when using survey data we use pweights, and sometimes when commands won't accept pweights we can use aweights because sometimes aweights and pweights will yield the same answer. The arithmetic mean is a good example of this. The much-loved **sum** command does not take pweights because it is meant to calculate sample characteristics and not inferential statistics. However, using aweights with **sum** will give you the same mean point estimate as using pweights with **mean**.

What aweights cannot do though, is sum properly to population totals. In this case, we must use pweights. Also ensure the weight variables you are using are indeed raising weights – meaning they are designed to sum to population totals. Not all pweights are raising weights and weights do not have to raise to population totals to be nationally representative. If weights are normalised, then they will yield nationally representative proportions, but not totals.

Let's continue working with **svy** and this time incorporate weights. Type:

```
mean numscore wscore
```

```
svyset [pw=weightfs]  
svy: mean numscore wscore  
estat effects, deff
```

Now you will notice that the point estimate of the mean changes, as do our measures of statistical uncertainty. Notice the DEFF is larger than when we added strata but much smaller than when we added clusters and more uniformly distributed across our two example variables. Why do weights also affect precision? Because by weighting some observations more or less this reduces the information content of the dataset (you are not getting equal information content from all observations). This is mainly owed to adjustment for unequal probability of selection and non-response.

Now, let's put all of these elements of the survey design together:

```
mean numscore wscore  
svyset imicscluster [pw=weightfs], strata(urban)  
svy: mean numscore wscore  
estat effects, deff
```

The full effect of accounting for sampling design is pretty important for accurate estimation. As you can hopefully tell from this exercise, sampling design can alter the point estimate of the mean and adjust the level of precision.

13. DESCRIBING DATA: PLOTS AND TABLES

Now that we have an outcome and treatment variable and understand enough about our sampling design to estimate accurately, let's start doing some actual estimation to describe our data. We are going to start with univariate descriptions and then turn to bivariate. Some of these commands we have covered, but now you are going to learn when is best to apply them. Let's start by describing our treatment variable, private. Type:

```
svy: mean private
```

We have run this command before but now you might want to think about the difference between running this for the sample in general, versus your sample of analysis. We have thought a lot about missing data, but we are not done yet! We are specifically interested in comparing private and numscore, so our analysis sample will be limited to those children who have non-missing data for *both* numscore and private. Compare the previous output to this:

```
svy: mean private if numscore != .
```

What you should immediately notice is our quite healthy sample size has taken a hit! This output is telling us that only about 64 thousand children have non-missing information for both. Is that right? You can use the **tab** command to explore schooltype when numscore is non-missing. You will see that there are about 16 000 children who have numscores but are NIU for numscores. How is this possible based on what we know about the non-missing schooltype sample. It turns out that there are about 16 000 kids who have numscores who have never attended school or who are not currently attending school! How might this be the case? Either way, we cannot compare the variation in these children's numscores to their school type so they can't be in our sample. How does this update your description of your sample or your understanding of your research question? Either way, it becomes useful to create an analysis sample dummy like the following to use with all your commands:

```
g mysample = numscore != . & private != . & age >= 7 & age <= 14  
tab mysample
```

It's possible you will have to rewrite this conditional statement over and over because you always have to apply it. If this is the case, you can decide to sub-set your data using the **keep** command. This command only keeps the observations that meet your specifications. An analogous command we've already come across is **drop**. Both **keep** and **drop** can drop variables as well as observations. Note that dropping observations can be dangerous if you don't have a record somewhere of how you got from the original dataset to this sub-set. It is also only appropriate to do this once you are focusing on a sub-set as opposed to exploring the dataset in general. This is why using do files is a good practice. Type:

```
keep if mysample == 1  
count
```

The number of observations is now much reduced, and you won't need to type **if mysample == 1** every time you run a command. You can think of this as your *analysis* sample which is different from the full sample. We spent some time thinking about how our dataset on our desktop looks (more or less) different to the true population in the real world. Now is a good time to reflect again about how your research decisions, research question framing, data cleaning decisions and data quality issues in general have added another layer of difference between the dataset on your desktop and the true population.

Let's turn to describing numscore. Since the numscore is a discrete continuous variable, a natural choice to see the distribution is a histogram, which we have encountered before. Recall the code is as follows:

```
hist numscore, discrete
```

Something to note about **hist** is that you cannot use weights (or, more correctly, you can only use frequency weights). If you try adding weights, you will get an error. We've just spent time thinking about why weights are important, so this is a blow. It turns out with some data wrangling one can create a weighted histogram using a weighted version of the **collapse** command, but we leave that for later.

Another option for plotting numeric data which allows weights is a kernel density plot which we can plot either just using **kdensity** or within the wrapper **tw** (twoway) which has lots of useful functionality. Try:

```
tw (kdensity numscore [aw=weightfs])
```

This plot does not look so great partly because a kernel density is more appropriate for more continuous data than numscore. But we can also smooth the data by controlling the bandwidth. Type:

```
tw (kdensity numscore [aw=weightfs], bwidth(1))
```

Smoothing can simplify plots and make them more readable, but a balance needs to be struck where you are not smoothing away important information. There are all sorts of other ways that you can adjust the formatting of **tw** that you can see in the **help** file. But by now, we have a pretty good idea of the basic distribution of our variables. These are not necessarily plots or tables that would appear in the main text of your research paper, but it is important you have a grip on what they look like. Furthermore, we are probably also interested in how they vary with other variables. Let's turn to bivariate descriptions.

The very first bivariate description we might be interested in is between our outcome and treatment. STATA offers many different ways to describe data and one skill a researcher must develop is to discern which descriptions maximise both information and readability. Consider the following two ways to learn about the covariance between numscore and private:

```
tab numscore [aw=weightfs], su(private)  
tab private [aw=weightfs], su(numscore)
```

How would you interpret these different sets of output? And which is most useful?

Again, the **tw** and kernel density can come in handy here. **Tw** is flexible enough to let you plot two plots on the same plot region. Type:

```
tw (kdensity numscore if private == 0 [aw=weightfs], bwidth(1)) ///  
(kdensity numscore if private == 1 [aw=weightfs], bwidth(1))
```

The `///` is do file notation used at the end of a line where the command runs onto the next line. It tells STATA to treat these two lines as one command. Consider another way to plot the same information where the plots are instead side-by-side. Type:

```
tw (kdensity numscore [aw=weightfs], bwidth(1)), by(private)
```

Great, so now we have an idea of how private and numscore vary, but you might wonder how these variables vary with other variables. Numeracy scores probably have a strong relationship with age. To illustrate a three-way covariance, you could type something like:

```
tw (kdensity numscore if private == 0 [aw=weightfs], bwidth(1)) ///  
(kdensity numscore if private == 1 [aw=weightfs], bwidth(1)), by(age)
```

This is interesting because it shows that private school advantage is more evident amongst younger children. Why can't you make a precise comparison? We do not have confidence intervals on here. There are user-written commands that will allow you to add confidence intervals to kernel densities, but these are quite complex and we leave them for later.

What about how private school attendance varies by country. Remember we can use options with a twoway **tab** to extract more useful statistics. Run the following and interpret the first cell in each. Which is the most useful comparison that you would want to include in a paper?

```
tab country private [aw=weightfs], col nofreq  
tab country private [aw=weightfs], cell nofreq  
tab country private [aw=weightfs], r nofreq
```

This is also not that helpful because private is a dummy, so we only need one column, not two. Let's explore a visual way to represent this information. A good choice for dummies is a bar graph. The basic syntax for **gr bar** is as follows:

```
gr bar (mean) private [pw=weightfs]
```

Hmm, that's not so great. It's not very informative. The power of **gr bar** is combining this options like **over()** and **by()**. An advantage of **gr bar** is also that it accepts weights. Let's plot country variation in private by typing:

```
gr bar (mean) private [pw=weightfs], over(country)
```

Ah, the labels are unreadable because they are overlapping so badly. There are two potential solutions. We can either adjust label orientation as follows:

```
numlabel, remove  
gr bar (mean) private [pw=weightfs], over(country, lab(angle(vertical)))
```

Note how the adjustment of the label happens *within* the **over()** bracket, since it is these labels we are controlling. Or, we can convert a vertically-orientated bar graph to a horizontally orientated one as follows:

```
gr hbar (mean) private [pw=weightfs], over(country)
```

There are many other ways you can manipulate and format this graph all detailed in the **help** file. Below is some more complicated code that shows how you can create a prettier plot and also sort the bars into descending order to make it even more digestible.

```
gr hbar (mean) private [pw=weightfs], over(country, sort(private)) scheme(plottig)  
ytitle(share in private school)
```

Again, adding a confidence interval to this is possible but a bit complicated for now.

These plots are useful but they only show one or two variables when we might be interested in a few. Let's try make a table of descriptive statistics like you might see in a journal article. One way that we could organise this table is by private schooling since this is our variable of interest.

Let's say we are interested in creating a table with numscore, urban, the household wealth index, and mother's education. Explore these variables in the dataset. What you will notice is they are all different types of variables (e.g. categorical, continuous) and so we need to think carefully about how to describe them appropriately. We decide to have columns for private school attendance so we can compare children in the sample who did and did not attend private school. Imagine a table like the below that we want to populate.

	Private school	Non-private school
Numeracy score (SD)		
Urban (%)		
Household wealth index (SD)		
...		

For now, we are going to focus on means and weighted proportions but worry less about confidence intervals. This is usually acceptable for descriptive statistics (but not for analysis). We can compute the statistics we want one at a time using **tab, sum()** which combines the functionality of both the **tab** and **sum** commands. Type:

```
tab private [aw=weightfs], sum(numscore)
```

This provides nice, readable output we can copy-paste into our table. Another way we can do this is by using the **sum** command with **bysort** to provide separate output by private school attendance. The advantage of doing it this way is we can **sum** several variables at the same time. Type:

```
bysort private: sum numscore urban wscore [aw=weightfs]
```

What is the mean for urban? It is a number greater than one which is a hint that this variable is not properly set up as a dummy for **summing**. Let's recode it and try again:

```
tab urban  
recode urban (2 = 0), g(urbandummy)  
bysort private: sum numscore urbandummy wscore [aw=weightfs]
```

Great, we can copy this content into our table. But now what about mother's education? This is an ordinal categorical variable. Let's have a look:

```
tab edlevelmom
```

With categorical variables like this, it is often important to report all the categories and make it clear in the table that they sum to 100%. There might be some categories you don't want to include though (with implications for sample reported). For example, if you excluded codes 97 and up, you would omit 57 people. With a table of descriptive

statistics like this one it is good practice to report sample size in the table, in which case the sample size needs to be the same for all the statistics. It's a good reminder to always be careful about exactly who is in your sample, and makes you confront how adding more and more variables to your analysis can strip away at sample size if patterns of missing data vary across different variables. This becomes an important research choice as well – balancing richness in terms of parameters with the health of your sample size.

Let's use **tab, sum()** to get useful table content. Type:

```
tab edlevelmom private if edlevelmom < 97 [aw=weightfs], col nofreq  
tab edlevelmom private if edlevelmom < 97
```

The first set of output gives us our table output and the second confirms that sample size is slightly smaller than what we computed for the previous variables. To be correct, we should recalculate the previous output as follows to ensure a common sample size:

```
bysort private: sum numscore urbandummy wscore if edlevelmom < 97  
[aw=weightfs]
```

In practice, this has a negligible effect on the actual statistics, but you can feel comfortable knowing you have done it correctly and it is just more consistent to use the same sample.

Numeracy scores are higher for those in private school. Can you say from this table alone that private schools produce better learning outcomes for numeracy? What else do you notice? Children attending private schools are also more likely to live in urban areas, live in richer households, and have more educated parents. In other words, the *type* of child that attends private could be different to one who doesn't. Maybe this type of child would get better numeracy scores regardless of whether they went to private school or not? How can we isolate the difference in numeracy scores that is owed only to private schooling? Think about this question for the next lab.

	Private school	Non-private school
Numeracy score (SD)	15.3 (5.8)	12.4 (7.2)
Urban (%)	62.4	32.1

Household wealth index (SD)	0.94 (1.19)	-0.04 (0.87)
Mother's education		
- Less than primary	19.9	33.0
- Primary	23.8	38.2
- Lower secondary	11.2	6.9
- Secondary or higher	9.0	2.6
- Secondary	20.8	15.1
- Tertiary	14.6	3.9
- Vocational	0.5	0.0
	100%	100%
N	10 134	54 010

LIST OF STATA COMMANDS INTRODUCED IN MODULE 1

use	isid	gen	hist	recode	rename
clear	tab	egen	count	label var	tw kdensity
browse	sum	numlabel	replace	label define	mean
describe	help	sort	drop	label values	svyset
lookfor	codebook	order	keep	forval	gr bar

Lab Module 2: Introduction to regression analysis in Stata

OBJECTIVES: By the end of this lab, you should be comfortable implementing regressions in STATA and analysing output.

In the first part of this module, we continue with the example and dataset from module 1: the difference in learning outcomes for students who attend private vs public school – does private schooling improve numeracy scores?

Let's first svyset:

```
svyset imicscluster [pw=weightfs], strata(urban)
```

To answer our question, we could just compare raw means:

```
svy: mean numscore, over(private)
```

Here, we observe that the mean numeracy score for privately schooled students is 2.79 points higher than that for students attending other school types. You can calculate this in the STATA with the **display** command. The **display** command is how you essentially use STATA as a calculator! Type:

```
display 15.577836-12.783942
```

But are we comparing 'like with like'? It is likely that students are differentially selected into private schooling on characteristics that also matter for numeracy score outcomes – for example, mothers' education. Other variables may also impact numeracy score, like the child's age. Regression allows us a way to consider these other factors. Put generally, regression is a tool that can characterise the relationship between an outcome variable and a treatment variable (here, private schooling), while holding other relevant factors constant, in order to quantify how the outcome is associated with exposure to the treatment.

1. RUNNING A REGRESSION

We're going to focus on Ghana in the examples that follow. Because this is a subpopulation in our dataset, we will need to restrict our regression. You have seen how to do this using **if**, but using **if** restrictions with **svy:** commands will often not produce correct variance estimates for subpopulations. To compute estimates for

subpopulations, it's better to use the **subpop()** option with **svy**. For this, we need to create a dummy variable:

```
gen ghana=(country==288)
```

Now, run a regression of numeracy score on private school:

```
svy, subpop(ghana): reg numscore private
```

The constant term shows the average numeracy score when private==0, i.e. the mean for students who do not attend private school.

The coefficient on private shows the additional score obtained for privately schooled children. Let's see how this coefficient changes when we attempt to address selection bias by adding controls to our regression. We will add control by control, starting with age.

First, check that the variable is coded as you would expect:

```
tab age, m
```

```
svy, subpop(ghana): reg numscore private age
```

Notice how the constant term has dropped dramatically – this is because it's now showing the average when age is 0. Because age = 0 is meaningless here, the constant itself is not substantively interpretable anymore. If you want a meaningful constant, you can centre the variable, e.g. `gen age7=age-7` (7 is the lowest age with numeracy score). Including age7 in your regression instead of age would give you a constant interpretable for age==7 and private==0.

The relationship between private school and numeracy score has become stronger after adding an age control. This will occur if older children perform better and are also less likely to be enrolled in private schools in our sample. Omitting age led to our estimate of the relationship between numeracy score and private schooling being biased *downwards*. Here is a check you can use when thinking about whether omitting a given variable will bias your results upwards or downwards:

Assume your regression is specified as `reg Y A and B` is an omitted variable. Positive (upwards) bias arises when the correlations between B with A and Y are in the same direction and negative (downward) bias when they are in opposite directions. Because older children are *less* likely to be enrolled in private school but *more* likely to score higher in the numeracy tests, omitting age lead to downward bias.

	A and B are positively correlated	A and B are negatively correlated
B is positively correlated with Y	Positive Bias	Negative Bias
B is negatively correlated with Y	Negative Bias	Positive Bias

If you could not observe the correlation between age and private school attendance, you would have to rely on results from other studies, or your ‘expert judgement’. Next, add a control for gender. Interpret the coefficient on female. Has the coefficient on private school changed? No, and this makes sense if gender is related to performance but not private schooling.

```
gen female = (sex == 2)
```

```
svy, subpop(ghana): reg numscore private age female
```

Next, run:

```
svy, subpop(ghana): reg numscore private age female i.edlevelmom
```

Notice that inserting **i.** before a categorical variable in your regression tells STATA that **edlevelmom** is not a continuous variable. STATA automatically assigns the first category of **edlevelmom** (here, ‘less than primary’) to be the base category. The coefficients on the other categories are interpreted in relation to the base. I.e. children whose mothers have completed primary education score 2 points higher on average than children with mothers who have less than primary.

You can ask STATA to use a different base category, by specifying the category, e.g. **ib20.edlevelmom**. In this case, it uses category with underlying value 20 (primary education) as the base category.

```
svy, subpop(ghana): reg numscore privateschool age female ib20.edlevelmom
```

Notice that the coefficient on the ‘unknown’ category yields the highest premium over primary. Does this seem a bit odd? I think we should inspect this further –

```
tab edlevelmom if ghana==1 & numscore!=. & privateschool!=.
```

Ah, there are only 3 children in this category! Therefore, we shouldn't read too much into this result. An option could be to exclude them from the regression, but because their sample size is so small, this shouldn't affect your other estimates much. If it does, you may need to dig further. Verify:

```
gen ghanamumnomiss=(ghana==1 & edlevelmom!=98) // a dummy variable for our subpop
```

```
svy, subpop(ghanamumnomiss): reg numscore private age female  
ib20.edlevelmom
```

What happened to the coefficient on private when you added mother's education level? Is this what you expected?

Lastly, let's add wealth quintile:

```
svy, subpop(ghanamumnomiss): reg numscore private age female  
ib20.edlevelmom i.windex5
```

What happens to your coefficient on private schooling now?

Note that you can also run regressions in STATA without the **svy:** prefix, but this does not allow you to account for stratification. If you have administrative data, for example, this would be preferable. Compare:

```
svy, subpop(ghanamumnomiss): reg numscore privateschool age female  
ib20.edlevelmom i.windex5
```

with

```
reg numscore privateschool age female ib20.edlevelmom i.windex5 if ghana==1  
& edlevelmom!=98 [w= weightfs], cluster(imicscluster)
```

Because the increase in precision gained from stratification rarely outweighs the loss in precision from clustering, we can see that we obtain very similar results by simply clustering our standard errors. Using **svy:** is always preferable if you have survey data with stratification, but straightforward **reg** is an alternative if your data do not need to be **svyset**.

2. INTERACTION TERMS IN REGRESSIONS

Sometimes we are interested in 'heterogeneous treatment effects', for example, does the relationship between private schooling and learning outcomes differ for boys vs. girls? Or, does the gender gap differ in private schools vs. public schools? To illustrate, we will use a simple regression controlling only for age and gender:

```
reg numscore private##female age if country==288 [aw=weightfs],  
cluster(imicscluster)
```

or

```
svy, subpop(ghana): reg numscore private##female age
```

How to interpret each coefficient:

1.private: Difference between private vs public among males (female = 0).

1.female: Difference between females vs. males in public schools (private = 0).

1.private#1.female: The additional difference for females in private schools, over and above the private–public difference for males, and the female–male difference in public schools. In other words: how much the private-school effect differs by gender.

In the regression we ran, is this a statistically significant difference? This answers our question if there is differential relationship between private schooling and numeracy score for girls.

But suppose we want to know if there is a gender gap in private schools? We can't answer this by looking at the output alone and need to use STATA's **test** command to test the significance of **1.female** plus **1.privateschool#1.female**. Why these two variables?

Here's why: You have the following output,

$$Y = \beta_0 + \beta_1(\text{private}) + \beta_2(\text{female}) + \beta_3(\text{private} \times \text{female}) + \beta_4(\text{age})$$

To answer our question (is there a gender gap in private schools), we would need to compare average outcomes for: Female private students vs male private students.

To get the average outcome of a female private student, we'd set **female=1** and **private=1**. In our equation, this gives us $Y_F = \beta_0 + \beta_1 + \beta_2 + \beta_3$.

To get the average outcome of a male private student, we'd set **female==0 and private==1**: $Y_M = \beta_0 + \beta_1$.

The gap would be $Y_F - Y_M = \beta_0 + \beta_1 + \beta_2 + \beta_3 - (\beta_0 + \beta_1) = \beta_2 + \beta_3$.

Therefore, we test:

```
test _b[1.female] + _b[1.private#1.female] = 0
```

Note that you can use the **, coeflegend** option with reg to know the names of the variables you need to test, e.g.

```
reg numscore private##female age if ghana==1 [aw=weightfs],  
cluster(imicscluster) coeflegend
```

or

```
svy, subpop(ghana): reg numscore private##female age, coeflegend
```

Next, we ask, what is the relationship between numscore and private schooling for females? For this, we need to compare **female==1 and private==1** against **female==1 and private==0**. Returning to the equation above, this equates to $\beta_0 + \beta_1 + \beta_2 + \beta_3 - (\beta_0 + \beta_2) = \beta_1 + \beta_3$, thus:

```
test _b[1.private] + _b[1.private#1.female] = 0
```

3. A NEW EXAMPLE: RETURNS TO EDUCATION

We are now pivoting to a different example to get more practice running regressions in STATA and interpreting output. Open the dataset data_glss7.dta.

We want to examine the relationship between wages earned and educational attainment.

We will start with a regression that considers the relationship between weekly wages and completing junior high school (JHS). Note that we are not using the svy command here, but all regressions can be implemented using

```
svy, subpop(): reg
```

We need to first create a dummy variable for highest level of completed education JHS or above:

```
gen jhs = edattain>=3 & edattain<=8
```

```
reg realwages_wk jhs if age>=25 & age<=65 [aw=weight_s], cluster(clust)
```

The age restriction is important here because we want to ensure the majority of people in our sample have completed their education. You can test the robustness of your results to slightly lowering or increasing the age restrictions (e.g. **if age>=25 & age<=60**).

Interpret the constant term and the coefficient on **jhs** from the regression above.

Let's add some controls:

```
reg realwages_wk jhs age female fathed_yrs if age>=25 & age<=65  
[aw=weight_s], cluster(clust)
```

What happens to the coefficient on **jhs**? Did it change in the way you would expect? You always want to ask yourself this question and hence the approach of adding controls one at a time can be useful.

Interpret the coefficients on female and father's years of education.

Perhaps you decide to control for mother's co-residency, and you notice that there is a statistically significant relationship between this variable and wages.

```
reg realwages_wk jhs age female fathed_yrs i.moth_res if age>=25 & age<=65  
[aw=weight_s], cluster(clust)
```

BUT, does this mean it should be in your regression? Not necessarily. We can have 'bad' controls. A rule of thumb is that if the variable is measuring something post-'treatment', it is a bad control. Current co-residency is a great example, because it is happening in a period after education has been completed - those more likely to be living with their parents now could do so out of necessity *because* they earn lower wages. Controlling for this 'disrupts' the causal chain between education and wages.

Similarly, why might you not want to control for occupation? Or simply add all variables that may be correlated with wage? Well, this is possible, but would answer a different question: "What is the return to education *holding occupation fixed*?". Occupation is an outcome itself of education. Ideally, in answering the return to education question, you'd only want to control for confounders: pre-treatment variables that affect both education and wages.

It is important to recognise that the decision about whether a control is good or bad is not always clear cut. It may differ depending on your research question, context, and motivation.

4. LOGGING A DEPENDENT VARIABLE

Often it is useful to log transform a continuous variable. This not only normalises the distribution of the variable (useful when distributions are very skewed, minimising the contribution of outliers), but allows us to measure proportional (percentage) change rather than absolute change. Let's log transform weekly wages and use it as our dependent variable:

```
gen logwklywage = log(realwages_wk)
```

```
reg logwklywage jhs female age fathed_yrs if age>=25 & age<=60 [aw=weight_s],  
cluster(clust)
```

Interpretation: the regression coefficient now gives the approximate **percentage** change in Y generated by changing the corresponding regressor. To get the exact % change: $(\exp(\beta_i) - 1) \times 100$.

```
di (exp(.4106686)-1)*100
```

This tells us that completing JHS or higher is associated with a 50.78% increase in weekly wages!

Rule of thumb: 1-unit increase in X results in an $[(\exp(\beta_i) - 1) \times 100]$ % change in Y . When β is less than about .2, $\exp(\beta) - 1$ and β are close enough to justify reference to β as percentage change⁴.

Word of caution: zeros cannot be logged, so be sure to check your variable before creating its log. A bad candidate for being logged would be test scores with zeros, since all children with a 0 score would be assigned a missing in the logged form.

5. EDUCATION AS A CONTINUOUS VARIABLE

Consider years of education as our variable of interest:

```
reg logwklywage edyrtotal female age if age>=25 & age<=65 [aw=weight_s],  
cluster(clust)
```

⁴ Angrist, J. D., & Pischke, J. S. (2014). Mastering'metrics: The path from cause to effect. Princeton university press.

Now we interpret the coefficient on education as the % change in wage when education increases by a year. It becomes harder to benchmark this as a big or small relationship. As we discussed in the lecture, you could: compare your result to previous estimates of the effect of schooling on wages, consider its size relative to the effect of other policies or to an established benchmark/goal, or contextualise the size relative to the mean wage in your sample, or to a change over time. You're essentially looking for some way of anchoring the estimate and putting its magnitude into context.

Notice that if you were to introduce an interaction term with years of education, you need to tell STATA it is continuous by prefixing **c.**, else when using the **##** operators, STATA automatically assumes it is categorical, e.g.

```
reg logwklywage c.edyrtotal##female age if age>=25 & age<=65 [aw=weight_s], cluster(clust)
```

Finally, let's use **educlvl** – a categorical variable. Run the following regression and interpret the results. Recall, coefficients on a categorical variable are interpreted relative to the base category, which Stata has automatically assigned to 'less than primary completed'. Again, notice how we need to use **i.** to let Stata know we have a categorical variable.

```
tab educlvl if age>=25 & age<=65
```

```
reg logwklywage i.educlvl age female fathed_yrs if age>=25 & age<=65 [aw=weight_s], cluster(clust)
```

You could also create a dummy variable for each education level, and include these in your regression. A useful way to do this is:

```
tab educlvl, gen(edlevel)
```

This creates a dummy variable for each category, as illustrated below:

```
. des edlevel*
```

Variable name	Storage type	Display format	Value label	Variable label
edlevel1	byte	%8.0g		educlvl==Less than primary completed
edlevel2	byte	%8.0g		educlvl==Primary completed
edlevel3	byte	%8.0g		educlvl==Secondary completed
edlevel4	byte	%8.0g		educlvl==University completed
edlevel5	byte	%8.0g		educlvl==NA
edlevel6	byte	%8.0g		educlvl==Missing

6. TRY IT YOURSELF

This exercise largely replicates the content above, so STATA code is not provided. Create the variables needed for your regression; a 'SHS or higher' education level indicator/dummy variable and log of wages. You will first need to examine the **edattain** variable to see which codes you should use to create 'SHS or higher'.

- a. Tabulate your 'SHS or higher' variable and consider the mean wage when your variable is 0 and 1. HINT: use the **tab** command with the **sum** option.
- b. Press regress - build up to your final wage regression by starting with a simple regression and adding more covariates. Use logged wage as your outcome variable.
- c. Interpret the coefficient on SHS+ completion in your final regression.
- d. Is the coefficient statistically significant? What about practically?
- e. Is the relationship you find robust to how you restrict your sample by age range?

7. SAME DATA, DIFFERENT QUESTION

Let's assume we have a research question about the association between starting school 'on time' (age 6 – or younger?) and total years of education completed.

Let's begin by plotting a graph of the mean years of completed education (**edyrtotal**) against age started school.

Check your variable **agestrtprim**:

```
tab agestrtprim
```

99% of children have started by the age of 11. There are no oddly coded missing values. We will restrict our scatter plot to those who started primary between the ages of 5 and 11. It seems unlikely that children would start primary before the age of 5, and these may be measurement error.

What happens if we just plot **edyrtotal** against **agestrtprim**?

```
scatter edyrtotal agestrtprim if agestrtprim>=5 & agestrtprim<=11
```

Hmm, is this telling us anything useful about the relationship? It would be more useful if we could observe *average* years of education completed for each **agestrtprim**.

So, we need to generate the mean years of completed education by age. We could do that by:

```
egen mean_edyrtotal =mean(edyrtotal), by(agestrtprim)
```

or

```
preserve  
collapse edyrtotal [w=weight_s] if age>=25, by(agestrtprim)  
list
```

If you run **preserve** at the same time as your following commands, once the code has finished running the original dataset will be restored. But, if you are running a full do-file, you will need to type `restore` to get your original dataset back.

Collapse creates a mini dataset where **edyrtotal** has now been replaced by its mean value, for each value of **agestrtprim**. **List** allows you to observe what the mini dataset looks like, before the data is automatically restored.

Why is **collapse** better than `egen` here to create a variable with the mean? Because we can apply the sampling weight. Try to collapse the data without first preserving your data and then browse the data to make sure that you understand how collapse is working.

We use **twoway scatter**, since as you have seen the **twoway** graphing function easily allows you to add more plots.

```
preserve  
collapse edyrtotal [w=weight_s] if age>=25, by(agestrtprim)  
twoway scatter edyrtotal agestrtprim if agestrtprim>=5 & agestrtprim<=11  
restore
```

***separating by gender**

```
Preserve  
collapse edyrtotal [w=weight_s], by(agestrtprim female)  
twoway ///
```

```
(scatter edyrtotal agestrtprim if agestrtprim>=5&agestrtprim<11&female==1) ///
(scatter edyrtotal agestrtprim if agestrtprim>=5&agestrtprim<11&female==0), ///
legend(order(1 "Female" 2 "Male")) ytitle("Mean years of schooling")
graph export "filepath\graphname.png"
restore
```

In the plot separated by gender, note how we have added a legend. The three lines `///` are also a way to run a command over multiple lines. We have also illustrated code to save your graph using **graph export**. Use **help graph export** to see what other formats graphs can be saved as,

In the scatter plot, we see there seems to be a negative relationship between age started school and years of complete education for adults.

Generate a dummy/indicator variable for starting school on time. Note how the code below assigns those who started before age 6 as 'on time'. You might want to make a different decision here, but in writing about your research it would be important to note what decision you've taken and why.

```
gen ontime = (agestrtprim<=6)

replace ontime = . if agestrtprim==.
```

Are the group who start on time similar on important dimensions related to schooling to those who don't? Cross tab with the row and column options here, e.g.:

```
tab ontime fatheduc if age>=25 [aw=weight_s], row
```

What information is the following command giving you?

```
tab ontime if age>=25 [aw=weight_s], sum(edyrtotal)
```

Regress completing your **ontime** on a JHS+ variable (i.e. your dependent variable is going to be a dummy variable):

```
gen jhs = edattain>=3 & edattain<=8 // created earlier in the module

reg jhs ontime female age fatheds_yrs if age>=25 [aw=weight_s], cluster(clust)
```

We interpret this as a percentage **point** change. Enrolling in school on time is associated with a 12.6 percentage point increase in completing JHS level or higher, holding age, gender and father's education constant. An ordinary least squares

regression with a dummy dependent variable is also called a linear probability model.

Note how we use language such as 'associated' and 'relationship' rather than 'impacts' or 'effects'. This is because we are signalling that we know this regression is unlikely to be showing a causal relationship between starting school on time and completing JHS or higher.

What is the relationship between ontime and completing JHS+ in percentage terms? You can sometimes compute the coefficient over the constant term:

```
di .1259052/ .6433815
```

But recall the constant is an average at age and fathers education at 0, and for men. Because age = 0 is meaningless here, the constant itself is usually not substantively interpretable. We can rather use margins:

```
margins ontime, atmeans
```

to get the predicted values associated with ontime vs. not ontime, setting all other variables to their mean values. Then we can compute the percentage change:

```
di (.7996277- .636563 )/ .636563
```

Sidebar: Why do you think we are restricting our sample by age? [As with returns to education, think about those who might still be completing their schooling]

Now, interact gender and ontime:

```
reg jhs ontime##female age fathed_yrs if age>=25 [aw=weight_s], cluster(clust)
```

How much less likely to complete JHS or higher are girls who do not start on time vs. boys who do not start on time? [HINT: you can get this straight from the regression output]

What is the overall relationship between completing JHS+ and starting school on time for girls?

```
di .0845716+.082681
```

Is the gender gap among on-time students statistically different from zero?

test _b[1.female] + _b[1.ontime#1.female] = 0

Another useful approach to interpreting interaction terms, in addition to the equation approach used above, is to use the margins post-estimation command, e.g.

reg jhs ontime##female age fathed_yrs if age>=25 [aw=weight_s], cluster(clust)

Linear regression

Number of obs = 13,210
F(5, 987) = 196.78
Prob > F = 0.0000
R-squared = 0.1210
Root MSE = .42527

(Std. err. adjusted for 988 clusters in clust)

jhs	Coefficient	Robust std. err.	t	P> t	[95% conf. interval]	
1.ontime	.0845716	.0133762	6.32	0.000	.0583224	.1108207
1.female	-.2160963	.0138798	-15.57	0.000	-.2433337	-.1888589
ontime#female 1 1	.082681	.0186207	4.44	0.000	.0461404	.1192216
age	-.0002227	.0004484	-0.50	0.619	-.0011027	.0006572
fathed_yrs	.0201762	.0011244	17.94	0.000	.0179697	.0223827
_cons	.6604845	.0257544	25.65	0.000	.609945	.7110241

margins ontime#female

. margins ontime#female

Predictive margins

Number of obs = 13,210

Model VCE: Robust

Expression: Linear prediction, predict()

	Delta-method Margin	std. err.	t	P> t	[95% conf. interval]	
ontime#female 0 0	.758381	.0098873	76.70	0.000	.7389784	.7777835
0 1	.5422846	.0122899	44.12	0.000	.5181674	.5664019
1 0	.8429525	.0092727	90.91	0.000	.8247561	.8611489
1 1	.7095372	.0116132	61.10	0.000	.6867478	.7323266

This gives you the predicted value of the outcome for each of these combinations. Look first at students who are not on time (ontime = 0): moving from female==0 to female==1 is the exact coefficient on **1.female** (.5422846-.758381=-.2160963).

Similarly, comparing males only: $.8429525 - .758381 = 0.0845715$ is exactly the marginal effect of ontime for males (**1.ontime**)

The interaction coefficient (Does the “on-time effect” differ for females compared with males?) can be computed as a **difference-in-differences**:

$$(\text{female \& ontime} - \text{female \& not ontime}) - (\text{male \& ontime} - \text{male \& not ontime}) \\ = .7095372 - .5422846 - (.8429525 - .758381).$$

You can visualise this as a table:

	Male (female=0)	Female (female=1)	Gender difference
Not on time (0)	0.7584	0.5423	-0.2161
On time (1)	0.8430	0.7095	-0.1334
On-time effect	+0.0846	+0.1673	Interaction = +0.0827

Put differently: $(0.0846 - 0.1673) = -0.2161 - (-0.1334) = 0.0827$

You don't need to worry too much about this, but it is useful to understand that margins lets us read interaction models as simple group averages first, and connect those averages back to the regression coefficients.

We can easily see if the predictions are statistically different or similar from each other by comparing the confidence intervals, i.e. the male-female difference in private schooling is statistically significant, because the confidence intervals from these two predictions don't overlap.

Use **help margins** to see its full capabilities, but use with caution – make sure you understand what output margins is generating and what is being compared to what!

Next, we want to see if this relationship between age started primary and educational attainment still holds when we use a different measure of educational attainment:

```
reg edyrtotal ontime age female fathed_yrs if age>=25 [aw=weight_s],  
cluster(clust)
```

What do we see?

What if we wanted someone's predicted years of education based on these characteristics?

```
predict xb
```

```
browse edyrtotal ontime age female fathed_yrs xb
```

Why might we want this? Predicted values tell us what the model implies for real people, whereas coefficients tell us how variables relate to each other. BUT, whether you consider predicted values will depend on your research question. You can also use the **margins** command to generate predicted values for specific values of other covariates. **Xb** is just a variable name, you could specify any varname that suits you.

We also might want to see if our results are robust to how parental education is specified. Try using categories of parental education (e.g. **fatheduc**) rather than years of education:

```
reg edyrtotal ontime age female i.fatheduc if age>=25 [aw=weight_s],  
cluster(clust)
```

EXPORTING RESULTS

What if you want to compare output across different regression specifications, or you want to present your results in a paper? **Outreg2** is a great and flexible command. It is user-written, so needs to be installed:

```
ssc install outreg2, replace
```

```
reg edyrtotal ontime age female if age>=25 [aw=weight_s], cluster(clust)
```

```
outreg2 using "[file path/results]", bdec(3) excel replace ctitle("reg1")
```

```
reg edyrtotal ontime age female fathed_yrs mothed_yrs if age>=25  
[aw=weight_s], cluster(clust)
```

```
outreg2 using "[file path/results]", bdec(3) excel append ctitle("reg2")
```

Outreg2 can also be used to extract **margins** output, but one then needs to use the **,post** option with your margins command. Note that the **append** option is used to add columns to your table of regression output.

As is always the case with Stata, there is more than one way to easily manage your regression output. An example of another command that creates excel tables from

your regression results is shown below. Another way to install user-written commands is as follows:

net search estout

Follow the onscreen prompts. Once you have installed **estout**, run the code below and then use **help** to explore some of the options.

```
reg edyrtotal ontime age female if age>=25 [aw=weight_s], cluster(clust)
```

```
estimates store ed1
```

```
reg edyrtotal ontime age female fathed_yrs mothed_yrs if age>=25  
[aw=weight_s], cluster(clust)
```

```
est store ed2
```

```
estout ed1 ed2 using filepath\results.xls
```

LIST OF STATA COMMANDS USED IN THIS LAB

estout	reg	display	graph export	predict
outreg2	collapse	predict	tab, gen()	browse
twoway	reg, coeflegend	ssc install	margins	restore
gen	test	net search	est store	preserve
scatter	svy: reg	help		

Module 3: Replication lab

This practical uses data from Duflo, E., Dupas, P., and Kremer, M. (2015). School governance, teacher incentives, and pupil–teacher ratios: Experimental evidence from Kenyan primary schools. *Journal of Public Economics*, 123, 92–110.

The study randomized schools to receive an extra contract teacher (ETP program) or to serve as the control group. Thereafter, half the schools in the ETP group were randomly selected to receive the SBM (school-based management) intervention. For this exercise, we will ignore the SBM intervention and just focus on the impact of being in an ETP program school.

To make it easier to find the relevant variables, we have created a dataset that contains only the variables that you will need for this practical. The dataset is called **rctprac1.dta**. We will share the do-file for this practical after you have attempted it, but it is included in this STATA deck because the ‘learning bits’ speak to relevant STATA commands.

Create a global for your working directory (A global macro in Stata is a named container for text or values that is stored in memory and can be called from any command or program during the session using **\$macroname**)

```
global wd "C:/Users/" // change to your own file path
```

Open the dataset

```
use "$wd/rctprac1.dta", clear
```

Question 1:

Use the **codebook** command (or another that makes sense you you) to answer:

- a. How many students are there in the dataset?
- b. How many different schools are there in the dataset?

Question 2:

One of the issues that the intervention hoped to address was the very large class sizes (or pupil-teacher ratios) in Kenyan schools.

- a. At baseline (before the intervention) what was the average class size?
- b. What is the largest class size?

- c. At baseline, what is the average class size for ETP schools and for control schools?
- d. Use a regression to test for balance between the two groups on class size. Are the differences significant?
- e. What level variable is class size?

LEARNING BIT: The problem with running a regression of treatment group on class size is that you have multiple observations per school that artificially increase the sample size and even small differences will be statistically significant. You want to let STATA know that you want to test for differences at the school level. How do you do this?

The following code will assign a value of 1 to the first observation (student) in each school and a 0 to all other learners in that school:

```
egen sch_one=tag(schoolid)
```

You can use this variable with an if statement to ask STATA to show you summary statistics at the school level (one observation per school) rather than at the learner level (one observation per learner). For example, if we wanted to know what percentage of schools were located in the Bungoma district, we would type:

```
sum district if sch_one==1
```

- f. Create the **sch_one** variable using the code above and use it to check if there is balance between the schools in the treatment and control groups on class size. Are the differences significant?
HINT: this is the same regression from (e), but restricted to one observation per school.

Question 3:

- a. What is the overall rate of attrition?
- b. Is there differential attrition between the treatment groups?
- c. Is the difference significant? (HINT: run a regression of treatment on attrition) How do you interpret the coefficient when your dependent variable is a dummy variable?

LEARNING BIT: Recall that unless we tell it otherwise, STATA (or other software) assumes our data are from a simple random sample where all the observations are independent. However, in this data set we have students within schools. Within a school there is likely correlation in the observables across students. To correct the standard errors to allow for correlation in the unobservables between students in the same school, we need to let STATA know that we have students clustered within schools. We do this by adding the cluster robust option to our regression. To test if there is differential attrition between learners in control and ETP schools, we amend the regression command as follows:

```
reg attrition etpschool, cluster(schoolid)
```

- d. Run the amended regression taking the clustering into account. What has happened to the point estimate? What has happened to the standard error?
- e. Is the overall attrition rate different for boys and girls?

LEARNING BIT: Beyond knowing whether there is differential attrition between the treatment and control groups, we are interested in whether the correlates of attrition differ between the two groups (i.e. did we lose different types of students from each group?). To investigate this, we need to use interaction terms. In the following example, we are examining whether attrition by gender differs between the treatment groups. Using a double hash is the recommended way of including an interaction term in STATA, because common post-estimation commands, e.g. **margins**, are able to correctly identify the interaction. STATA automatically assumes the variables in the interaction are categorical or dummy variables, unless you tell it otherwise. In this case, you would use **c.** before the variable.

```
reg attrition etpschool##girl, cluster(schoolid)  
margins etpschool#girl
```

```
reg attrition etpschool##c.agetest, cluster(schoolid)
```

- f. What do the results from the regression (1) above tell us about attrition in this study?

Question 4:

We are now going to look at class size at the endline by treatment group.

- a. What is the average class size at endline for control schools and for ETP schools?

b. Use the help function to find out how to create side-by-side histograms of endline class size for control and treatment schools. Alternatively, use the graphic menu to look for this option. HINT: **, by()**

Question 5:

In addition to very large class sizes, heterogeneity within a class is another issue in Kenya. There is considerable variation in age within a class a result of late entry to school and grade repetition.

- a. What is the range in ages? What is the interquartile range?
- b. How many people who were tested at endline have missing age data?

LEARNING BIT: If we want to use age as a covariate in our regression, those students with missing values will be dropped from the regression. Even though there are not many students with missing age data, we can land up dropping a lot of observations when we add several covariates with missing data to our regression. This potentially introduces selection bias. A common strategy to deal with this is to create a dummy variable indicating that **agetest** is missing and then assigning a fixed value (e.g. 0) for **agetest** for all those that are missing. We then include both **agetest** and the missing variable in the regression.

Create a dummy variable that indicates that **agetest** is missing.

```
gen agetest_m=agetest>=. if attrition==0
```

Create a new variable **agetest_c** which is equal to **agetest** if **agetest** is not missing and equal to zero if **agetest** is missing

```
recode agetest (.=0) if attrition==0, gen(agetest_no_miss)
```

Question 6:

LEARNING BIT: We will now estimate the impact on the standardized total score of being in an ETP school. To get correct standard errors we need to let STATA know that we randomized at the school level (i.e the school is the cluster) and we need to include dummy variable(s) for the strata. Before randomizing the sample was stratified by district. Use the **describe** or **codebook** commands to inspect the district variable. What is the variable type (e.g. byte, long, string)? It is a string variable and will generate an error if you try and include it in your regression. To control for strata in your regression you will need to create a numeric variable (e.g. byte) for district. Use the following command to generate a dummy variable for the districts:

tab district, gen(temp)

This command creates a dummy variable for each category in district. In this case there are two districts, and the command creates temp1 and temp2. Rename these temporary variables to the district names.

rename temp1 bungoma
rename temp2 butere

- a. Estimate the impact on the standardized total score of being in an ETP school. Remember to cluster your standard errors and include a dummy variable for the strata. Interpret your regression results.
- b. Add the following covariates to your regression: **girl, kcpe2004**
agetest_no_miss agetest_m. What happens to the standard error of the estimated treatment effect? Why?

Question 7:

We will now turn our focus to the comparison between contract and govt teachers in ETP schools

- a. Within ETP schools, estimate the impact on the standardized total score on being assigned to a contract teacher. Interpret the results. HINT: you will need to use an **if** statement in your regression command.
- b. We are interested in whether there are heterogenous effects based on learner proficiency at baseline. Rerun your regression including an interaction term between being assigned to a contract teacher and the

indicator that the student was in the bottom half of their class at baseline.
Fully interpret the results.

- c. **test** whether the overall effect of ETP teachers is different from 0 for students in the bottom half

LIST OF STATA COMMANDS USED IN THIS LAB

codebook	reg
egen	reg ##
histogram	reg,coeflegend
recode	test
sum	global
tab, sum()	tab, gen()
help	sum, detail